

You Can't Block My C2: It's Your Google Calendar

Red Team Village · DEFCON 34



› whoami

Nishant Tayade

@0xNirvana

Blue Team → Red Team

INTUIT



Origin of This Research

Prior Art

- APT-related Threat Intelligence
- Not a new idea
- Red Team Research

The Gap

- No Mythic-native solution
- No SSH into custom relay
- No bespoke server

Design Goals

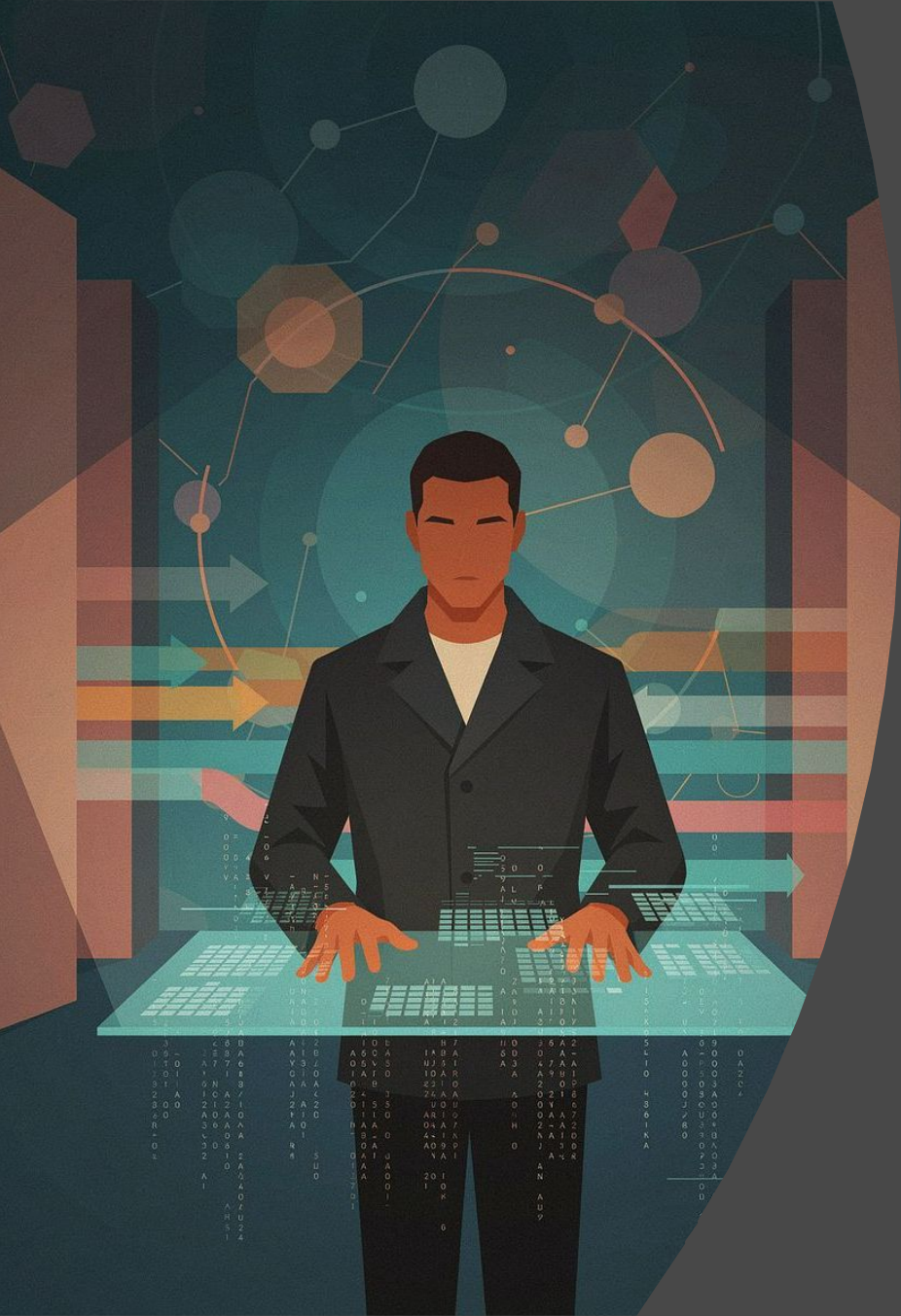
- Install like any other Mythic Agent/Profile
- Build payloads from the Mythic UI without manual steps
- Open source

What Was Built

Chronos: the Mythic agent

Epoch: the Mythic C2 profile

 **Note:** Names matter.



Why Google Calendar?

Why Calendar Is Uniquely Suited



Allowlisted Almost Everywhere

googleapis.com is on every enterprise proxy allowlist.



Traffic Blends Naturally

Endpoints already sync Calendar; it's ambient noise to the SOC.



Blocking = Business Outage

Blocking Calendar API stops enterprise scheduling; blue teams can't pull that lever.



This is hard to block and easy to miss but not "undetectable forever."

Architecture



Epoch vs. Chronos

Dimension	Epoch	Chronos
Role	Mythic C2 profile (Docker relay)	Mythic agent (Python payload)
Job	Shuttle encrypted blobs via Calendar	Poll, execute, return results
Crypto	Never decrypts i.e. dumb pipe	Mythic AES-256-CBC + HMAC
Talks to	Mythic + Google Calendar API	Google Calendar API only

Epoch is to Calendar what an HTTP profile is to HTTP, just the transport layer. Chronos holds the keys and does the work.



Dead-drop C2

- No direct communication
- Shared Google Calendar provides async read/write dead drop
- Legitimate SaaS API traffic only

Communication Flow

Chronos → Calendar
write check-in / result event

Chronos polls Calendar
decrypt → execute → loop

Epoch polls Calendar → Mythic
forward opaque blob as agent_message

Mythic → Epoch → Calendar
send next instruction



📌 **Google Calendar** is the dead-drop hub at the center of every loop

Where Bytes Hide

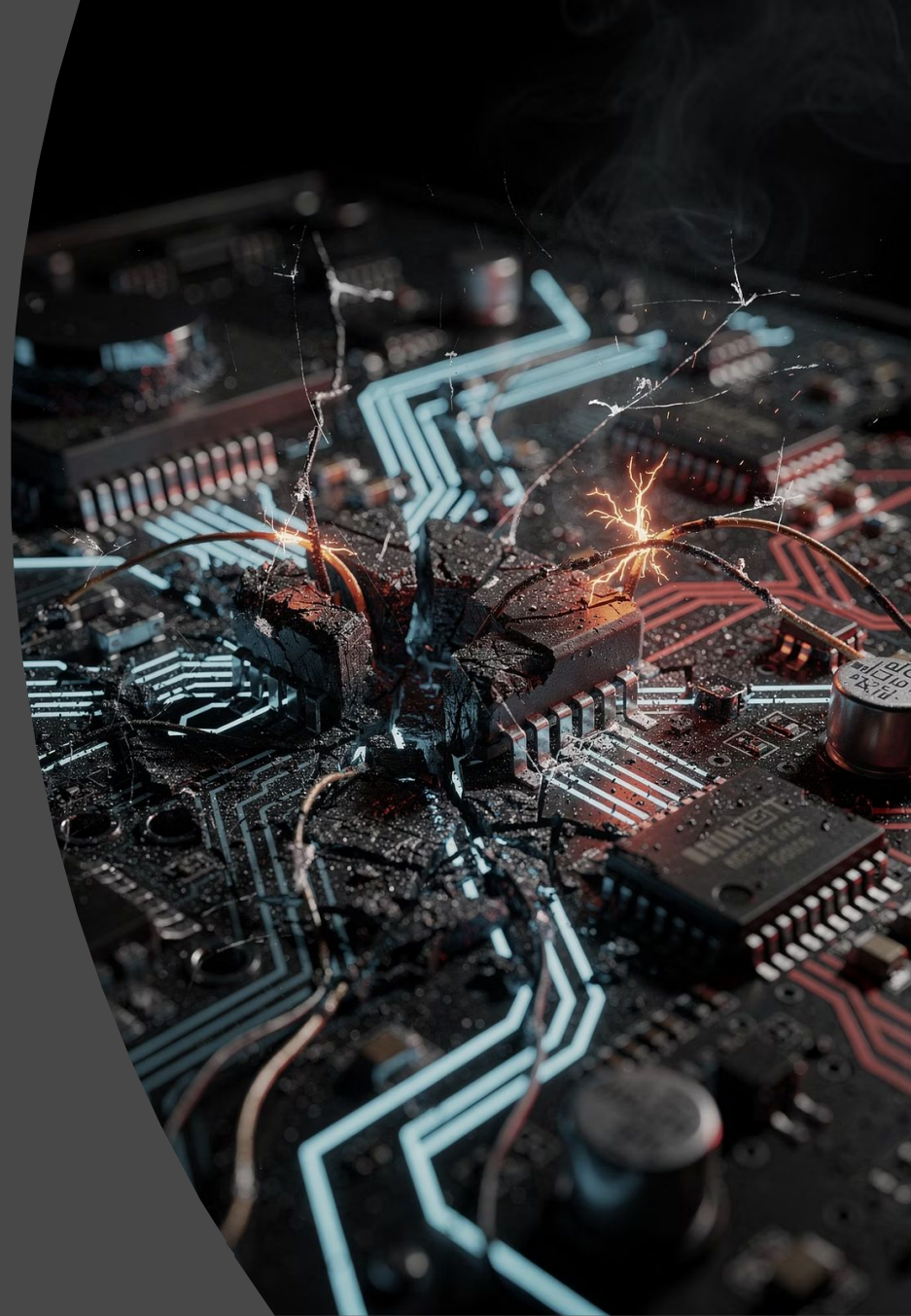
- **Routing:** `extendedProperties.private` — `agent_id`, `msg_type`, `proto_version`, `message_id`
- **Payload:** event `description` — Mythic-encrypted payload (base64 = **encoding**, not encryption)
- **Agent crypto boundary:** callback keys live in **Chronos** meaning Calendar / Workspace readers see metadata + blobs, not your shell commands
- **Multi-agent:** one calendar can run multiple isolated agents identified by `agent_id`

Message Types:

msg_type	Direction & meaning
checkin	Chronos → Epoch: agent registers; Mythic assigns callback
tasking	Chronos → Epoch: agent asks for work
cmd	Epoch → Chronos: Mythic tasking / replies (agent polls for these)
resp	Chronos → Epoch: command output back to Mythic

DEMO

What Broke



Honest Constraints

30–120s

Round-Trip Latency

Not HTTP-snappy. Dominated by Calendar poll intervals and eventual visibility.

~500KB

Practical File Size

Single-shot transfer cap. Not a data exfil pipe.

10–20

Agents per Calendar

Calendar API quota caps chatty multi-agent deployments.

⚠ This is a **command channel**, not an interactive shell or data exfil pipe. If you need megabytes out or sub-second loops, use a different channel.

Calendar API Visibility Delays

- Event created $\neq$ immediately visible to other API clients
- Often 30–60s before list queries see it
- Before hardening: check-in failed ~50% under tight timeouts
- Fix: `max(240s, poll_interval×12)` agent wait + Epoch retry on check-in path
- Result: 100% cold-start check-in in lab (poll_interval 5–15s)



The Empty-Response Heuristic Saga

Clever Optimization

Built a size heuristic: if Mythic's response looks "empty," skip writing a Calendar event. Saves API quota.

1

Silent Failure

Operators saw no output. Looked exactly like an agent crash or network drop which is not a relay logic bug.

2

3

4

Three Breakages

Encrypted blob size drifts with padding. Baseline shifted. Real tasks were silently classified as "empty" and dropped.

Fix: Delete the Heuristic

Ripped it out entirely. Always relay. Heuristics on opaque ciphertext are dangerous by construction.

⊗ Rule: if you cannot decrypt it, you cannot guess whether it is empty. **Always relay.**

Building C2 on SaaS APIs

These failures apply to anyone wrapping Drive, Sheets, Teams, Notion, or any other SaaS API as a C2 transport.



SaaS Is Not a Transport Protocol

No push. No delivery guarantees. Eventual consistency. Design for that reality from day one.



Test the Unhappy Path

Timeouts, duplicates, partial visibility, API quota exhaustion. Happy-path testing will fool you every time.

Operate It. Detect It.



When to Use Calendar C2 (and When Not To)

✓ Use it when

- Primary beacon burned
- Re-entry, liveness checks, light async tasking
- Low-and-slow: recon, staging, credential work batched into poll windows
- Long dwell, minimal noise

✗ Skip it when

- Sub-second interactive shells
- Large file exfil (~500 KB cap in this build)
- No Google path (locked-down Workspace, no API)
- Time-sensitive tasks

Defender's Perspective

Google Workspace Audit Logs

- Calendar API audit logs via Workspace Admin console
- Most SOC stacks don't ingest these by default
- Start here

Rapid Create / Delete Cycles

- High-churn event creation/deletion is not normal Calendar behavior
- Baseline your org's Calendar API shape; the anomaly will stand out

extendedProperties.private Usage

- Legitimate apps rarely spam this field
- Repeated writes across many events = strong signal worth alerting on

DPI / Network Layer

- Unusual volume or timing of googleapis.com from non-browser hosts (servers, odd subnets)
- Pair with identity and audit logs

Takeaways

1 Allowlisted SaaS APIs are a structural blind spot

- Allowlists create cover
- Defenders watching domain/IP egress will miss channels that look like normal SaaS sync

2 Dead drops work but reliability engineering is the hard part

- The party trick is trivial
- Building a reliable relay on an eventually-consistent, quota-bound API is the real engineering

3 Blue teams: monitor API patterns, not just egress domains

- Watching "is it Google?" is not enough
- Baseline Calendar API behavior: event churn, field usage, service account origins
- Alert on deviations

Next Up



[**github.com/OxNirvana/mythic-epoch-chronos**](https://github.com/OxNirvana/mythic-epoch-chronos)

Epoch (C2 Profile) + Chronos (Agent)

[**linkedin.com/in/nishant-tayade**](https://www.linkedin.com/in/nishant-tayade)